

Documentation

Contenu

[introduction](#)

[installation](#)

[ajax](#)

[déclencheurs](#)

[modificateurs de déclenchement](#)

[filtres de déclenchement](#)

[événements spéciaux](#)

[vote](#)

[interrogation de charge](#)

[indicateurs](#)

[cibles](#)

[échange](#)

[synchronisation](#)

[transitions CSS](#)

[échanges hors bande](#)

[paramètres](#)

[confirmant](#)

[héritage](#)

[booster](#)

[WebSockets et SSE](#)

[histoire](#)

[demandes et réponses](#)

[validation](#)

[animations](#)

[extensions](#)

[événements et journalisation](#)

[débogage](#)[script](#)[attribut hx-on](#)[intégration tierce](#)[Composants Web](#)[mise en cache](#)[sécurité](#)[configuration](#)

htmx en bref

htmx est une bibliothèque qui permet d'accéder aux fonctionnalités modernes des navigateurs directement depuis le HTML, sans passer par JavaScript.

Pour comprendre le HTML, commençons par examiner une balise d'ancrage :

```
<a href="/blog">Blog</a>
```

Cette balise d'ancrage indique au navigateur :

« Lorsqu'un utilisateur clique sur ce lien, envoyez une requête HTTP GET à '/blog' et chargez le contenu de la réponse dans la fenêtre du navigateur. »

Dans cette optique, considérez l'extrait de code HTML suivant :

```
<button hx-post="/clicked"
  hx-trigger="click"
  hx-target="#parent-div"
  hx-swap="outerHTML">
  Click Me!
</button>
```

Ceci indique à htmx :

« Lorsqu'un utilisateur clique sur ce bouton, envoyez une requête HTTP POST à '/clicked' et utilisez le contenu de la réponse pour remplacer l'élément par l'identifiant parent-div dans le DOM. »

htmx étend et généralise l'idée de base du HTML en tant qu'hypertexte, ouvrant ainsi de nombreuses possibilités supplémentaires directement au sein du langage :

- Désormais, n'importe quel élément, et pas seulement les liens et les formulaires, peut émettre une requête HTTP.

- Désormais, n'importe quel événement, et pas seulement les clics ou les soumissions de formulaires, peut déclencher des requêtes.
- Désormais, n'importe quel [verbe HTTP](#) , et pas seulement `GET` et `POST` , peut être utilisé.
- Désormais, n'importe quel élément, et non plus seulement la fenêtre entière, peut être la cible d'une mise à jour par la requête.

Notez que lorsque vous utilisez htmx, le serveur répond généralement au *format HTML* et non *JSON* . Cela vous permet de rester fidèle au [modèle de programmation web original](#) , en utilisant [l'hypertexte comme moteur de l'état de l'application](#), sans même avoir besoin de comprendre pleinement ce concept.

Il convient de mentionner que, si vous préférez, vous pouvez utiliser le [data-](#) préfixe lors de l'utilisation de htmx :

```
<a data-hx-post="/click">Click Me!</a>
```

Si vous comprenez les concepts liés à htmx et souhaitez découvrir les particularités de la bibliothèque, veuillez consulter notre page [PARTICULARITÉS](#) .

Guide de migration de la version 1.x à la version 2.x

La [version 1](#) de htmx est toujours prise en charge et compatible avec IE11, mais la dernière version de htmx est la 2.x.

Si vous migrez de htmx 1.x vers [htmx 2.x](#) , veuillez consulter le [guide de migration htmx 1.x](#).

Si vous migrez vers htmx depuis intercooler.js, veuillez consulter le [guide de migration d'intercooler](#) .

Installation

Htmx est une bibliothèque JavaScript sans dépendances, conçue pour les navigateurs. Son utilisation est donc aussi simple que d'ajouter une `<script>` balise `<script>` à l'en-tête de votre document. Aucun système de compilation n'est nécessaire.

Via un CDN (par exemple jsDelivr)

La méthode la plus rapide pour utiliser htmx est de le charger via un CDN. Il vous suffit d'ajouter ceci à votre balise `<head>` et c'est parti !

```
<script src="https://cdn.jsdelivr.net/npm/htmx.org@2.0.10/dist/htmx.min.js" integrity=
```

Une version non minifiée est également disponible :

```
<script src="https://cdn.jsdelivr.net/npm/htmx.org@2.0.10/dist/htmx.js" integrity="sha256-1234567890" ></script>
```

Bien que l'approche CDN soit extrêmement simple, vous pourriez envisager de [ne pas utiliser de CDN en production](#) .

Télécharger une copie

La méthode suivante la plus simple pour installer htmx consiste simplement à le copier dans votre projet.

Téléchargez-le `htmx.min.js` [depuis jsDelivr](#) et ajoutez-le au répertoire approprié de votre projet, puis incluez-le là où c'est nécessaire avec une `<script>` balise :

```
<script src="/path/to/htmx.min.js"></script>
```

npm

Pour les systèmes de construction de type npm, vous pouvez installer htmx via [npm](#) :

```
npm install htmx.org@2.0.10
```

Après l'installation, vous devrez utiliser les outils appropriés `node_modules/htmx.org/dist/htmx.js` (ou `.min.js`). Par exemple, vous pouvez intégrer htmx à des extensions et à du code spécifique à votre projet.

Webpack

Si vous utilisez webpack pour gérer votre javascript :

- Installez- `htmx` le via votre gestionnaire de paquets préféré (comme npm ou yarn).
- Ajoutez l'importation à votre `index.js`

```
import 'htmx.org';
```

Si vous souhaitez utiliser la `htmx` variable globale (recommandé), vous devez l'injecter dans la portée de la fenêtre :

- Créer un fichier JS personnalisé
- Importez ce fichier dans votre `index.js` (en dessous de l'importation de l'étape 2)

```
import 'path/to/my_custom.js';
```

- Ajoutez ensuite ce code au fichier :

```
window.htmx = require('htmx.org');
```

- Enfin, reconstruisez votre paquet

AJAX

Le cœur du format htmx est un ensemble d'attributs qui vous permettent d'émettre des requêtes AJAX directement depuis le HTML :

Attribut	Description
hx-get	Envoie une GET requête à l'URL indiquée.
hx-post	Envoie une POST requête à l'URL indiquée.
hx-put	Envoie une PUT requête à l'URL indiquée.
patch HX	Envoie une PATCH requête à l'URL indiquée.
suppression hx	Envoie une DELETE requête à l'URL indiquée.

Chacun de ces attributs prend une URL pour envoyer une requête AJAX. L'élément enverra une requête du type spécifié à l'URL donnée lorsqu'il sera [déclenché](#) .

```
<button hx-put="/messages">
  Put To Messages
</button>
```

Cela indique au navigateur :

Lorsqu'un utilisateur clique sur ce bouton, envoyez une requête PUT à l'URL /messages et chargez la réponse dans le bouton.

Déclenchement des requêtes

Par défaut, les requêtes AJAX sont déclenchées par l'événement « naturel » d'un élément :

- `input` , `textarea` et `select` sont déclenchés par l' `change` événement
- `form` est déclenché par l' `submit` événement
- Tout le reste est déclenché par l' `click` événement.

Si vous souhaitez un comportement différent, vous pouvez utiliser l' attribut [hx-trigger](#) pour spécifier quel événement déclenchera la requête.

Voici un élément `div` qui envoie une requête `/mouse_entered` lorsqu'une souris le survole :

```
<div hx-post="/mouse_entered" hx-trigger="mouseenter">
  [Here Mouse, Mouse!]
</div>
```

Modificateurs de déclenchement

Un déclencheur peut également comporter quelques modificateurs supplémentaires qui en modifient le comportement. Par exemple, si vous souhaitez qu'une requête ne s'exécute qu'une seule fois, vous pouvez utiliser le `once` modificateur suivant pour le déclencheur :

```
<div hx-post="/mouse_entered" hx-trigger="mouseenter once">
  [Here Mouse, Mouse!]
</div>
```

Voici d'autres modificateurs que vous pouvez utiliser pour les déclencheurs :

- `changed` - N'émettre une requête que si la valeur de l'élément a changé.
- `delay:<time interval>` Attendez le délai indiqué (par exemple `1s`) avant d'envoyer la requête. Si l'événement se déclenche à nouveau, le compte à rebours est réinitialisé.
- `throttle:<time interval>` Attendez le délai imparti (par exemple `1s`) avant d'envoyer la requête. `delay` Si un nouvel événement survient avant l'expiration de ce délai, il sera ignoré ; la requête sera donc déclenchée à la fin de la période impartie.
- `from:<CSS Selector>` Écoutez l'événement sur un autre élément. Cela peut servir, par exemple, pour les raccourcis clavier. Notez que ce sélecteur CSS n'est pas réévalué en cas de changement de page.

Vous pouvez utiliser ces attributs pour mettre en œuvre de nombreux modèles d'expérience utilisateur courants, tels que [la recherche active](#) :

```
<input type="text" name="q"
  hx-get="/trigger_delay"
  hx-trigger="keyup changed delay:500ms"
  hx-target="#search-results"
  placeholder="Search...">
<div id="search-results"></div>
```

Cette entrée enverra une requête 500 millisecondes après un événement de relâchement de touche si l'entrée a été modifiée et insérera les résultats dans le `div` avec l'identifiant `search-results`.

Plusieurs déclencheurs peuvent être spécifiés dans l'attribut `hx-trigger`, séparés par des virgules.

Filtres de déclenchement

Vous pouvez également appliquer des filtres de déclenchement en utilisant des crochets après le nom de l'événement, encadrant une expression JavaScript qui sera évaluée. Si l'expression est vraie, `true` l'événement sera déclenché ; sinon, il ne le sera pas.

Voici un exemple qui se déclenche uniquement lors d'un clic avec la touche Ctrl enfoncée sur l'élément.

```
<div hx-get="/clicked" hx-trigger="click[ctrlKey]">
  Control Click Me
</div>
```

Les propriétés de ce type `ctrlKey` seront d'abord résolues par rapport à l'événement déclencheur, puis par rapport à la portée globale. Le `this` symbole sera défini sur l'élément actuel.

Événements spéciaux

htmx fournit quelques événements spéciaux à utiliser dans `hx-trigger` :

- `load` - Se déclenche une fois lors du premier chargement de l'élément.
- `revealed` - Se déclenche une fois lorsqu'un élément apparaît pour la première fois dans la fenêtre d'affichage.
- `intersect` - Se déclenche une seule fois lorsqu'un élément entre pour la première fois dans la zone d'affichage. Deux options supplémentaires sont prises en charge :
 - `root:<selector>` - un sélecteur CSS de l'élément racine pour l'intersection
 - `threshold:<float>` - un nombre à virgule flottante compris entre 0,0 et 1,0, indiquant le niveau d'intersection sur lequel déclencher l'événement.

Vous pouvez également utiliser des événements personnalisés pour déclencher des requêtes si vous avez un cas d'utilisation avancé.

Vote

Si vous souhaitez qu'un élément interroge l'URL donnée plutôt que d'attendre un événement, vous pouvez utiliser la `every` syntaxe avec l' `hx-trigger` attribut :

```
<div hx-get="/news" hx-trigger="every 2s"></div>
```

Cela indique à htmx

Toutes les 2 secondes, effectuez une requête GET à `/news` et chargez la réponse dans la div

Si vous souhaitez interrompre l'interrogation d'une réponse du serveur, vous pouvez répondre avec le code de réponse HTTP [286](#) et l'élément annulera l'interrogation.

Interrogation de charge

Une autre technique permettant de réaliser un sondage en HTML est le « sondage par chargement », où un élément spécifie un `load` déclencheur ainsi qu'un délai, et se remplace par la réponse :

```
<div hx-get="/messages"
    hx-trigger="load delay:1s"
    hx-swap="outerHTML">
</div>
```

Si le `/messages` point de terminaison continue de renvoyer un élément `div` configuré de cette manière, il continuera à interroger l'URL toutes les secondes.

L'interrogation de la charge peut être utile dans les situations où une interrogation a un point final auquel l'interrogation se termine, par exemple lorsque vous affichez une [barre de progression](#) à l'utilisateur .

Indicateurs de requête

Lorsqu'une requête AJAX est émise, il est souvent judicieux d'informer l'utilisateur qu'une opération est en cours, car le navigateur ne lui fournira aucun retour visuel. Vous pouvez réaliser cela en HTML en utilisant `htmx-indicator` une classe.

La `htmx-indicator` classe est définie de telle sorte que l'opacité de tout élément possédant cette classe soit de 0 par défaut, le rendant invisible mais présent dans le DOM.

Lorsqu'HTML effectue une requête, il attribue une `htmx-request` classe à un élément (soit l'élément requérant, soit un autre élément, si spécifié). Cette `htmx-request` classe provoque `htmx-indicator` une transition d'opacité à 1 pour l'élément enfant qui la possède, affichant ainsi l'indicateur.

```
<button hx-get="/click">
  Click Me!
  
</button>
```

Voici un bouton. Lorsqu'on clique dessus, une `htmx-request` classe lui est ajoutée, ce qui révèle l'élément GIF de chargement. (J'aime bien [les animations de chargement SVG](#) en ce moment.)

Bien que la `htmx-indicator` classe utilise l'opacité pour masquer et afficher l'indicateur de progression, si vous préférez un autre mécanisme, vous pouvez créer votre propre transition CSS comme ceci :

```
.htmx-indicator{
  display:none;
}
```

```
.htmx-request .htmx-indicator{
  display:inline;
}
.htmx-request.htmx-indicator{
  display:inline;
}
```

Si vous souhaitez `htmx-request` ajouter la classe à un élément différent, vous pouvez utiliser l'attribut `hx-indicator` avec un sélecteur CSS :

```
<div>
  <button hx-get="/click" hx-indicator="#indicator">
    Click Me!
  </button>
  
</div>
```

Ici, nous désignons explicitement l'indicateur par son identifiant. Notez que nous aurions `div` également pu appliquer la classe à l'élément parent, ce qui aurait donné le même résultat.

Vous pouvez également ajouter l'attribut `disabled` aux éléments pour la durée d'une requête en utilisant l'attribut `hx-disabled-elt`.

Cibles

Si vous souhaitez que la réponse soit chargée dans un élément différent de celui qui a effectué la requête, vous pouvez utiliser l'attribut `hx-target`, qui accepte un sélecteur CSS. Reprenons notre exemple de recherche en direct :

```
<input type="text" name="q"
  hx-get="/trigger_delay"
  hx-trigger="keyup delay:500ms changed"
  hx-target="#search-results"
  placeholder="Search...">
<div id="search-results"></div>
```

Vous pouvez constater que les résultats de la recherche seront chargés dans `div#search-results`, plutôt que dans la balise d'entrée.

Sélecteurs CSS étendus

`hx-target` et la plupart des attributs qui acceptent un sélecteur CSS prennent en charge une syntaxe CSS « étendue » :

- Vous pouvez utiliser le `this` mot-clé, qui indique que l'élément sur lequel `hx-target` se trouve l'attribut est la cible
- Cette `closest <CSS selector>` syntaxe trouvera l'élément ancêtre **le plus proche**, ou l'élément lui-même, correspondant au sélecteur CSS donné. (Par exemple,

`closest` `tr` elle ciblera la ligne de tableau la plus proche de l'élément.)

- Cette `next <CSS selector>` syntaxe permettra de trouver l'élément suivant dans le DOM correspondant au sélecteur CSS donné.
- Cette `previous <CSS selector>` syntaxe permettra de trouver l'élément précédent dans le DOM correspondant au sélecteur CSS donné.
- `find <CSS selector>` Cette fonction trouvera le premier élément enfant descendant correspondant au sélecteur CSS donné (par exemple, `find tr` elle ciblera la première ligne enfant descendante de l'élément).

De plus, un sélecteur CSS peut être enveloppé dans des caractères `<` et `>`, imitant la syntaxe [littérale de requête](#) de l'hyperscript.

Les cibles relatives comme celle-ci peuvent être utiles pour créer des interfaces utilisateur flexibles sans surcharger votre DOM d' `id` attributs.

Échange

htmx propose plusieurs méthodes pour remplacer le code HTML renvoyé dans le DOM. Par défaut, le contenu remplace le `<head>` `innerHTML` de l'élément cible. Vous pouvez modifier ce comportement à l'aide de l'attribut `hx-swap` avec l'une des valeurs suivantes :

Nom	Description
<code>innerHTML</code>	Par défaut, le contenu est placé à l'intérieur de l'élément cible.
<code>outerHTML</code>	remplace l'élément cible entier par le contenu renvoyé
<code>afterbegin</code>	ajoute le contenu avant le premier enfant à l'intérieur de la cible
<code>beforebegin</code>	ajoute le contenu avant la cible dans l'élément parent de la cible
<code>beforeend</code>	ajoute le contenu après le dernier enfant à l'intérieur de la cible
<code>afterend</code>	ajoute le contenu après la cible dans l'élément parent de la cible
<code>delete</code>	supprime l'élément cible quelle que soit la réponse
<code>none</code>	n'ajoute pas le contenu de la réponse (les échanges hors bande et les en-têtes de réponse seront toujours traités)

Échanges de morphing

Outre les mécanismes d'échange classiques décrits ci-dessus, htmx prend également en charge les échanges *avec morphing*, via des extensions. Ces échanges avec morphing tentent d' *intégrer* le nouveau contenu au DOM existant, plutôt que de simplement le remplacer. Ils préservent souvent mieux des éléments tels que le focus,

l'état vidéo, etc., en modifiant les nœuds existants sur place pendant l'opération d'échange, au prix d'une consommation accrue de ressources CPU.

Les extensions suivantes sont disponibles pour les échanges de style morphing :

- [Idiomorph](#) - Un algorithme de morphing créé par les développeurs de htmx.
- [Morphdom Swap](#) - Basé sur [morphdom](#) , la bibliothèque de morphing DOM originale.
- [Alpine-morph](#) - Basé sur le plugin [Alpine Morph](#) , il fonctionne parfaitement avec Alpine.js

Transitions de vue

La nouvelle API expérimentale [View Transitions](#) permet aux développeurs de créer des transitions animées entre différents états du DOM. Encore en développement et non disponible sur tous les navigateurs, elle est compatible avec htmx qui propose une solution de repli sans transition si l'API n'est pas prise en charge par un navigateur donné.

Vous pouvez expérimenter cette nouvelle API en utilisant les approches suivantes :

- Configurez la `htmx.config.globalViewTransitions` variable de configuration pour `true` utiliser les transitions pour tous les échanges.
- Utilisez l' `transition:true` option dans l' `hx-swap` attribut
- Si une transition d'échange d'éléments doit avoir lieu en raison de l'une des configurations ci-dessus, vous pouvez intercepter l' `htmx:beforeTransition` événement et `preventDefault()` l'utiliser pour annuler la transition.

Les transitions d'affichage peuvent être configurées à l'aide de CSS, comme indiqué dans [la documentation Chrome relative à cette fonctionnalité](#) .

Vous pouvez voir un exemple de transition de vue sur la page [Exemples d'animation](#) .

Options d'échange

L' attribut `hx-swap` offre de nombreuses options pour ajuster le comportement d'échange de balises htmx. Par exemple, par défaut, htmx remplace le titre d'une balise `title` présente n'importe où dans le nouveau contenu. Vous pouvez désactiver ce comportement en définissant le `ignoreTitle` modificateur sur `true`.

```
<button hx-post="/like" hx-swap="outerHTML ignoreTitle:true">Like</button>
```

Les modificateurs disponibles `hx-swap` sont :

Option	Description
transition	true ou false s'il faut utiliser l'API de transition de vue pour cet échange
swap	Le délai d'échange à utiliser (par exemple 100ms) entre le moment où l'ancien contenu est effacé et celui où le nouveau contenu est inséré.
settle	Le délai de stabilisation à utiliser (par exemple 100ms) entre l'insertion du nouveau contenu et sa stabilisation.
ignoreTitle	Si cette option est activée true , tout titre trouvé dans le nouveau contenu sera ignoré et le titre du document ne sera pas mis à jour.
scroll	top ou bottom , fera défiler l'élément cible vers le haut ou vers le bas
show	top ou bottom , fera défiler le haut ou le bas de l'élément cible jusqu'à ce qu'il soit visible.

Tous les modificateurs d'échange apparaissent après la spécification du style d'échange et sont séparés par des deux-points.

Consultez la documentation [hx-swap](#) pour plus de détails sur ces options.

Synchronisation

Il est souvent nécessaire de coordonner les requêtes entre deux éléments. Par exemple, on peut vouloir qu'une requête d'un élément remplace celle d'un autre, ou qu'elle attende la fin de la requête de l'autre.

htmx propose un [hx-sync](#) attribut pour vous aider à y parvenir.

Considérons une condition de concurrence entre la soumission d'un formulaire et la requête de validation d'un champ individuel dans ce code HTML :

```
<form hx-post="/store">
  <input id="title" name="title" type="text"
    hx-post="/validate"
    hx-trigger="change">
  <button type="submit">Submit</button>
</form>
```

Sans utiliser `hx-sync` , le fait de remplir les champs et de soumettre immédiatement le formulaire déclenche deux requêtes parallèles vers `/validate` et `/store` .

L'utilisation de `hx-sync="closest form:abort"` cette méthode sur l'élément input permettra de surveiller les requêtes sur le formulaire et d'annuler la requête de l'élément input si une requête de formulaire est présente ou démarre pendant que la requête de l'élément input est en cours :

```
<form hx-post="/store">
  <input id="title" name="title" type="text"
    hx-post="/validate"
    hx-trigger="change"
    hx-sync="closest form:abort">
  <button type="submit">Submit</button>
</form>
```

Cela résout la synchronisation entre les deux éléments de manière déclarative.

htmx prend également en charge une méthode programmatique pour annuler les requêtes : vous pouvez envoyer l' `htmx:abort` événement à un élément pour annuler toutes les requêtes en cours :

```
<button id="request-button" hx-post="/example">
  Issue Request
</button>
<button onclick="htmx.trigger('#request-button', 'htmx:abort')">
  Cancel Request
</button>
```

Vous trouverez davantage d'exemples et de détails sur la [hx-sync page des attributs](#).

Transitions CSS

htmx simplifie l'utilisation [des transitions CSS](#) sans JavaScript. Prenons l'exemple du contenu HTML suivant :

```
<div id="div1">Original Content</div>
```

Imaginez que ce contenu soit remplacé par du HTML via une requête Ajax avec ce nouveau contenu :

```
<div id="div1" class="red">New Content</div>
```

Notez deux choses :

- L'élément `div` possède le *même* identifiant dans le contenu original et dans le nouveau contenu.
- La `red` classe a été ajoutée au nouveau contenu

Dans ce cas, nous pouvons écrire une transition CSS de l'ancien état au nouvel état :

```
.red {
  color: red;
  transition: all ease-in 1s ;
}
```

Lorsque htmx intègre ce nouveau contenu, il le fera de telle sorte que la transition CSS s'applique au nouveau contenu, vous offrant ainsi une transition fluide et agréable vers le nouvel état.

En résumé, pour utiliser les transitions CSS pour un élément, il suffit de maintenir sa `id` stabilité entre les requêtes !

Vous pouvez consulter les [exemples d'animation](#) pour plus de détails et des démonstrations en direct.

Détails

Pour comprendre comment les transitions CSS fonctionnent réellement dans htmx, il faut comprendre le modèle d'échange et de stabilisation sous-jacent utilisé par htmx.

Lorsqu'un nouveau contenu est reçu d'un serveur, avant d'être intégré, le contenu existant de la page est analysé afin d'identifier les éléments correspondant à l'`id` attribut. Si une correspondance est trouvée, les attributs de l'ancien contenu sont copiés sur le nouvel élément avant l'intégration. Le nouveau contenu est alors intégré, mais avec les *anciennes* valeurs d'attributs. Enfin, les nouvelles valeurs d'attributs sont appliquées après un délai de stabilisation (20 ms par défaut). Un peu complexe, mais c'est ce qui permet aux transitions CSS de fonctionner sans que le développeur n'ait à écrire de JavaScript.

Échanges hors bande

Si vous souhaitez insérer directement du contenu provenant d'une réponse dans le DOM à l'aide de l'`id` attribut, vous pouvez utiliser l'attribut `hx-swap-oob` dans le code HTML *de la réponse* :

```
<div id="message" hx-swap-oob="true">Swap me directly!</div>  
Additional Content
```



Dans cette réponse, `div#message` l'élément serait directement inséré dans l'élément DOM correspondant, tandis que le contenu supplémentaire serait inséré dans la cible de manière normale.

Vous pouvez utiliser cette technique pour « intégrer » des mises à jour à d'autres requêtes.

Tables problématiques

Les éléments de tableau peuvent poser problème lorsqu'ils sont combinés à des échanges hors bande, car, selon la spécification HTML, beaucoup ne peuvent pas être utilisés seuls dans le DOM (par exemple `<tr>` ou `<td>`).

Pour éviter ce problème, vous pouvez utiliser une `template` balise pour encapsuler ces éléments :

```
<template>
  <tr id="message" hx-swap-oob="true"><td>Joe</td><td>Smith</td></tr>
</template>
```

Sélection du contenu à échanger

Si vous souhaitez sélectionner un sous-ensemble du code HTML de la réponse à insérer dans la cible, vous pouvez utiliser l'attribut `hx-select`, qui prend un sélecteur CSS et sélectionne les éléments correspondants dans la réponse.

Vous pouvez également sélectionner des éléments de contenu pour un échange hors bande en utilisant l'attribut `hx-select-oob`, qui prend une liste d'identifiants d'éléments à sélectionner et à échanger.

Préservation du contenu lors d'un échange

Si vous souhaitez conserver un contenu lors de changements de système (par exemple, un lecteur vidéo que vous souhaitez continuer à lire même en cas de changement de système), vous pouvez utiliser l'attribut `hx-preserve` sur les éléments que vous souhaitez conserver.

Paramètres

Par défaut, un élément à l'origine d'une requête inclura sa valeur, le cas échéant. S'il s'agit d'un formulaire, il inclura les valeurs de tous ses champs de saisie.

Comme pour les formulaires HTML, l'attribut `name` de l'élément `input` est utilisé comme nom de paramètre dans la requête envoyée par htmx.

De plus, si l'élément provoque une `GET` requête non valide, les valeurs de toutes les entrées du formulaire associé seront incluses (généralement, il s'agit du formulaire englobant le plus proche, mais cela pourrait être différent si, par exemple, `<form> <button form="associated-form">` est utilisé).

Si vous souhaitez inclure les valeurs d'autres éléments, vous pouvez utiliser l'attribut `hx-include` avec un sélecteur CSS de tous les éléments dont vous souhaitez inclure les valeurs dans la requête.

Si vous souhaitez filtrer certains paramètres, vous pouvez utiliser l'attribut `hx-params`.

Enfin, si vous souhaitez modifier les paramètres par programmation, vous pouvez utiliser l'événement `htmx:configRequest`.

Téléchargement de fichier

Si vous souhaitez téléverser des fichiers via une requête HTML, vous pouvez définir l'attribut `hx-encoding` `multipart/form-data` sur . Cela utilisera un `FormData` objet pour soumettre la requête, qui inclura correctement le fichier dans la requête.

Notez que, selon votre technologie côté serveur, vous devrez peut-être traiter les requêtes avec ce type de contenu de manière très différente.

Notez que htmx déclenche `htmx:xhr:progress` périodiquement un événement basé sur l' `progress` événement standard lors du téléchargement, auquel vous pouvez vous connecter pour afficher la progression du téléchargement.

Consultez la [section des exemples](#) pour découvrir des modèles de formulaires plus avancés, notamment [les barres de progression](#) et [la gestion des erreurs](#) .

Valeurs supplémentaires

Vous pouvez inclure des valeurs supplémentaires dans une requête en utilisant les attributs `hx-vals` (paires nom-expression au format JSON) et `hx-vars` (paires nom-expression séparées par des virgules qui sont calculées dynamiquement).

Confirmation des demandes

Il est souvent nécessaire de confirmer une action avant d'envoyer une requête. Le format HTML prend en charge l' `hx-confirm` attribut `confirmation`, qui permet de confirmer une action à l'aide d'une simple boîte de dialogue JavaScript :

```
<button hx-delete="/account" hx-confirm="Are you sure you wish to delete your account:
Delete My Account
</button>
```

L'utilisation d'événements permet de mettre en œuvre des boîtes de dialogue de confirmation plus sophistiquées. L' [exemple de confirmation](#) montre comment utiliser la bibliothèque [sweetalert2](#) pour la confirmation d'actions HTML.

Confirmation des requêtes à l'aide d'événements

Une autre option pour effectuer la confirmation consiste à utiliser l' [htmx:confirm événement](#) . Cet événement est déclenché à *chaque fois* qu'une requête est initiée (et pas seulement sur les éléments possédant un `hx-confirm` attribut) et peut être utilisé pour implémenter une confirmation asynchrone de la requête.

Voici un exemple d'utilisation [de SweetAlert](#) sur un élément possédant un `confirm-with-sweet-alert='true'` attribut :

```
document.body.addEventListener('htmx:confirm', function(evt) {
  if (evt.target.matches("[confirm-with-sweet-alert='true']")) {
    evt.preventDefault();
    swal({
      title: "Are you sure?",
      text: "Are you sure you are sure?",
      icon: "warning",
      buttons: true,
      dangerMode: true,
    }).then((confirmed) => {
      if (confirmed) {
        evt.detail.issueRequest();
      }
    });
  }
});
```

```
}  
});
```

Héritage des attributs

La plupart des attributs HTML sont hérités : ils s'appliquent à l'élément auquel ils appartiennent ainsi qu'à ses éléments enfants. Cela permet de « remonter » les attributs dans le DOM afin d'éviter la duplication de code. Prenons l'exemple du code HTML suivant :

```
<button hx-delete="/account" hx-confirm="Are you sure?">  
  Delete My Account  
</button>  
<button hx-put="/account" hx-confirm="Are you sure?">  
  Update My Account  
</button>
```

Nous avons ici un `hx-confirm` attribut dupliqué. Nous pouvons le déplacer vers un élément parent :

```
<div hx-confirm="Are you sure?">  
  <button hx-delete="/account">  
    Delete My Account  
  </button>  
  <button hx-put="/account">  
    Update My Account  
  </button>  
</div>
```

Cet `hx-confirm` attribut s'appliquera désormais à tous les éléments utilisant HTML qu'il contient.

Il arrive que l'on souhaite annuler cet héritage. Imaginons que nous ayons un bouton « Annuler » pour ce groupe, mais que nous ne souhaitions pas de confirmation. Nous pourrions alors ajouter une `unset` directive comme ceci :

```
<div hx-confirm="Are you sure?">  
  <button hx-delete="/account">  
    Delete My Account  
  </button>  
  <button hx-put="/account">  
    Update My Account  
  </button>  
  <button hx-confirm="unset" hx-get="/">  
    Cancel  
  </button>  
</div>
```

Les deux boutons du haut afficheraient alors une boîte de dialogue de confirmation, mais pas le bouton d'annulation du bas.

L'héritage peut être désactivé élément par élément et attribut par attribut à l'aide de l' `hx-disinherit` attribut.

Si vous souhaitez désactiver complètement l'héritage des attributs, vous pouvez définir la `htmx.config.disableInheritance` variable de configuration sur `true`. Cela désactivera l'héritage par défaut et vous permettra de le spécifier explicitement avec l' `hx-inherit` attribut `attribute`.

Boosting

Htmx permet de « booster » les ancres et formulaires HTML classiques grâce à l'attribut `hx-boost`. Cet attribut convertit toutes les balises d'ancrage et tous les formulaires en requêtes AJAX qui, par défaut, ciblent le corps de la page.

Voici un exemple :

```
<div hx-boost="true">
  <a href="/blog">Blog</a>
</div>
```

La balise d'ancrage dans cette div enverra une GET requête AJAX `/blog` et insérera la réponse dans la `body` balise.

Amélioration progressive

L'une de ses caractéristiques `hx-boost` est sa capacité à se dégrader progressivement si JavaScript est désactivé : les liens et les formulaires restent fonctionnels, mais sans utiliser de requêtes AJAX. Ce mécanisme, appelé **amélioration progressive**, permet à un public plus large d'accéder aux fonctionnalités de votre site.

D'autres modèles HTML peuvent également être adaptés pour obtenir une amélioration progressive, mais ils nécessiteront une réflexion plus approfondie.

Prenons l'exemple [de la recherche active](#). Tel qu'il est écrit, il ne fonctionnera pas correctement : une personne n'ayant pas activé JavaScript ne pourra pas utiliser cette fonctionnalité. Ce choix est fait par souci de simplicité, afin de rendre l'exemple aussi concis que possible.

Vous pouvez toutefois encapsuler le champ de saisie enrichi HTML dans un élément de formulaire :

```
<form action="/search" method="POST">
  <input class="form-control" type="search"
    name="search" placeholder="Begin typing to search users..."
    hx-post="/search"
    hx-trigger="keyup changed delay:500ms, search">
```

```
    hx-target="#search-results"
    hx-indicator=".htmx-indicator">
</form>
```

Grâce à cette solution, les clients compatibles JavaScript bénéficieraient toujours d'une expérience utilisateur optimale avec la recherche active, tandis que les clients non compatibles JavaScript pourraient utiliser la touche Entrée pour lancer une recherche. Mieux encore, il serait possible d'ajouter un bouton « Rechercher ». Il faudrait alors mettre à jour le formulaire avec un `hx-post` élément reprenant l' `action` attribut, ou éventuellement utiliser `hx-boost` une propriété spécifique.

Il faudrait vérifier côté serveur l' `HX-Request` en-tête pour faire la distinction entre une requête pilotée par HTML et une requête classique, afin de déterminer exactement ce qu'il faut afficher au client.

D'autres modèles peuvent être adaptés de la même manière pour répondre aux besoins d'amélioration progressive de votre application.

Comme vous pouvez le constater, cela exige davantage de réflexion et de travail. Cela exclut également certaines fonctionnalités. Ces compromis doivent être faits par vous, le développeur, en fonction des objectifs de vos projets et de votre public cible.

L'[accessibilité](#) est un concept étroitement lié à l'amélioration progressive. L'utilisation de techniques d'amélioration progressive, telles que `hx-boost` celles mentionnées ci-dessus, rendra votre application HTML plus accessible à un large éventail d'utilisateurs.

Les applications basées sur htmx sont très similaires aux applications web classiques, non pilotées par AJAX, car htmx est orienté HTML.

Par conséquent, les recommandations habituelles en matière d'accessibilité HTML s'appliquent. Par exemple :

- Utilisez autant que possible du HTML sémantique (c'est-à-dire les bonnes balises pour les bonnes choses).
- Assurez-vous que l'état de focus est clairement visible.
- Associez des étiquettes de texte à tous les champs du formulaire
- Optimisez la lisibilité de votre application grâce à des polices appropriées, un contraste adéquat, etc.

WebSockets et SSE

Les protocoles WebSocket et Server-Sent Events (SSE) sont pris en charge via des extensions. Veuillez consulter les pages des [extensions SSE](#) et [WebSocket](#) pour en savoir plus.

Soutien historique

Htmx fournit un mécanisme simple pour interagir avec l' [API d'historique du navigateur](#) :

Si vous souhaitez qu'un élément donné affiche son URL de requête dans la barre de navigation du navigateur et ajoute l'état actuel de la page à l'historique du navigateur, incluez l' attribut [hx-push-url](#) :

```
<a hx-get="/blog" hx-push-url="true">Blog</a>
```

Lorsqu'un utilisateur clique sur ce lien, htmx capture un instantané du DOM actuel et le stocke avant d'effectuer une requête vers /blog. Il effectue ensuite l'échange et ajoute une nouvelle adresse à l'historique.

Lorsqu'un utilisateur clique sur le bouton Retour, htmx récupère l'ancien contenu depuis le cache et le réintègre à la cible, simulant ainsi un retour à l'état précédent. Si l'emplacement est introuvable dans le cache, htmx effectue une requête AJAX à l'URL spécifiée, avec l'en-tête `HX-History-Restore-Request` défini sur « true », et attend en retour le code HTML nécessaire pour la page entière. Il est recommandé de toujours définir `htmx.config.historyRestoreAsHxRequest` l'en-tête sur « false » afin d'empêcher `HX-Request` son utilisation, ce qui permet ensuite de répondre avec des fragments de page. À l'inverse, si la `htmx.config.refreshOnHistoryMiss` variable de configuration est définie sur « true », un rechargement complet du navigateur est effectué.

REMARQUE : Si vous ajoutez une URL à l'historique, vous **devez** pouvoir y accéder et récupérer la page complète ! Un utilisateur peut copier-coller l'URL dans un e-mail ou l'ouvrir dans un nouvel onglet. De plus, le script HTML aura besoin de la page entière pour restaurer l'historique si celle-ci n'est pas présente dans le cache.

Spécifier l'élément d'instantané historique

Par défaut, htmx utilise l' `body` élément `<script>` pour créer et restaurer l'historique. C'est généralement le comportement approprié, mais si vous souhaitez utiliser un élément plus spécifique pour la création de l'historique, vous pouvez spécifier un autre élément à l'aide de l'attribut `hx-history-elt` .

Attention : cet élément devra figurer sur toutes les pages, sinon la restauration à partir de l'historique ne fonctionnera pas correctement.

Annulation des modifications du DOM par des bibliothèques tierces

Si vous utilisez une bibliothèque tierce et souhaitez utiliser l'historique HTML, vous devrez nettoyer le DOM avant la prise d'un instantané. Prenons l'exemple de la bibliothèque [TomSelect](#) , qui enrichit considérablement l'expérience utilisateur des éléments select. Configurons TomSelect pour transformer tout élément input doté de la `.tomselect` classe appropriée en un élément select enrichi.

Nous devons d'abord initialiser les éléments qui possèdent la classe dans le nouveau contenu :

```
htmx.onLoad(function (target) {
  // find all elements in the new content that should be
  // an editor and init w/ TomSelect
  var editors = target.querySelectorAll(".tomselect")
    .forEach(elt => new TomSelect(elt))
});
```

Cela créera un sélecteur enrichi pour tous les éléments d'entrée possédant la `.tomselect` classe. Cependant, cela modifie le DOM et nous ne souhaitons pas que cette modification soit enregistrée dans le cache d'historique, car TomSelect sera réinitialisé lorsque le contenu de l'historique sera rechargé à l'écran.

Pour remédier à cela, nous devons intercepter l' `htmx:beforeHistorySave` événement et supprimer les mutations TomSelect en `destroy()` les appelant :

```
htmx.on('htmx:beforeHistorySave', function() {
  // find all TomSelect elements
  document.querySelectorAll('.tomselect')
    .forEach(elt => elt.tomselect.destroy()) // and call destroy() on them
});
```

Cela rétablira le DOM à son état HTML d'origine, permettant ainsi d'obtenir un instantané propre.

Désactivation des instantanés de l'historique

Il est possible de désactiver la sauvegarde de l'historique pour une URL en définissant l'attribut `hx-history` `false` sur `true` sur n'importe quel élément du document courant, ou sur n'importe quel fragment HTML chargé dans le document courant par `htmx`. Ceci permet d'empêcher l'enregistrement de données sensibles dans le cache local, ce qui est important pour les ordinateurs partagés ou publics. La navigation dans l'historique fonctionnera normalement, mais lors de la restauration, l'URL sera interrogée auprès du serveur et non plus à partir du cache local.

Demands et réponses

Htmx attend des réponses HTML à ses requêtes AJAX, généralement des fragments HTML (bien qu'un document HTML complet, associé à une balise `hx-select`, puisse également convenir). Htmx insérera ensuite le code HTML reçu dans le document cible, selon la stratégie d'insertion spécifiée.

Parfois, vous pourriez ne rien vouloir faire dans l'échange, mais tout de même déclencher un événement côté client ([voir ci-dessous](#)).

Dans ce cas, par défaut, vous pouvez renvoyer un `204 - No Content` code de réponse, et htmx ignorera le contenu de la réponse.

En cas de réponse d'erreur du serveur (par exemple une erreur 404 ou 501), htmx déclenchera l' `htmx:responseError` événement que vous pourrez gérer.

En cas d'erreur de connexion, l' `htmx:sendError` événement sera déclenché.

Configuration de la gestion des réponses

Vous pouvez configurer le comportement décrit ci-dessus pour htmx en modifiant ou en remplaçant le `htmx.config.responseHandling` tableau. Cet objet est une collection d'objets JavaScript définis comme suit :

```
responseHandling: [
  {code:"204", swap: false}, // 204 - No Content by default does nothing, but
  {code:"[23]..", swap: true}, // 200 & 300 responses are non-errors and are swa
  {code:"[45]..", swap: false, error:true}, // 400 & 500 responses are not swapp
  {code:"...", swap: false} // catch all for any other response code
]
```

Lorsqu'HTMX reçoit une réponse, il parcourt le `htmx.config.responseHandling` tableau et vérifie si la `code` propriété d'un objet donné, interprétée comme une expression régulière, correspond au code de la réponse. Si c'est le cas, cette correspondance détermine le traitement de la réponse.

Les champs disponibles pour la configuration de la gestion des réponses sur les entrées de ce tableau sont :

- `code` - une chaîne de caractères représentant une expression régulière qui sera testée par rapport aux codes de réponse.
- `swap` - `true` si la réponse doit être insérée dans le DOM, `false` sinon
- `error` - `true` si htmx doit traiter cette réponse comme une erreur
- `ignoreTitle` - `true` si le HTML doit ignorer les balises de titre dans la réponse
- `select` - Un sélecteur CSS à utiliser pour sélectionner le contenu de la réponse
- `target` - Un sélecteur CSS spécifiant une cible alternative pour la réponse
- `swapOverride` - Un mécanisme d'échange alternatif pour la réponse

Exemples de configuration de la gestion des réponses

À titre d'exemple d'utilisation de cette configuration, prenons le cas où un framework côté serveur renvoie une `422 - Unprocessable Entity` réponse en cas d'erreur de validation. Par défaut, htmx ignorera cette réponse, car elle correspond à l'expression régulière `[45]..`.

En utilisant le mécanisme [de méta-configuration](#) pour configurer `responseHandling`, nous pourrions ajouter la configuration suivante :

```
<!--
  * 204 No Content by default does nothing, but is not an error
  * 2xx, 3xx and 422 responses are non-errors and are swapped
  * 4xx & 5xx responses are not swapped and are errors
  * all other responses are swapped using "..." as a catch-all
-->
<meta
  name="htmx-config"
  content='{
    "responseHandling":[
      {"code":"204", "swap": false},
      {"code":"[23]..", "swap": true},
      {"code":"422", "swap": true},
      {"code":"[45]..", "swap": false, "error":true},
      {"code":"...", "swap": true}
    ]
  }'
/>
```

Si vous souhaitez tout échanger, quel que soit le code de réponse HTTP, vous pouvez utiliser cette configuration :

```
<meta name="htmx-config" content='{ "responseHandling": [{"code": ".*", "swap": true}] }'
```

Enfin, il convient d'envisager l'utilisation de l'extension [Response Targets](#) , qui permet de configurer de manière déclarative le comportement des codes de réponse via des attributs.

CORS

Lorsque vous utilisez htmx dans un contexte inter-origines, n'oubliez pas de configurer votre serveur Web pour définir les en-têtes Access-Control afin que les en-têtes htmx soient visibles côté client.

- [Access-Control-Allow-Headers](#) (pour les en-têtes de requête)
- [Access-Control-Expose-Headers](#) (pour les en-têtes de réponse)

[Consultez tous les en-têtes de requête et de réponse implémentés par htmx.](#)

En-têtes de requête

htmx inclut un certain nombre d'en-têtes utiles dans les requêtes :

En-tête	Description
HX-Boosted	indique que la requête passe par un élément utilisant hx-boost

En-tête	Description
HX-Current-URL	L'URL actuelle du navigateur
HX-History-Restore-Request	« vrai » si la requête concerne la restauration de l'historique après une perte dans le cache d'historique local
HX-Prompt	la réponse de l'utilisateur à une invite hx
HX-Request	Toujours « vrai » sauf pour les requêtes de restauration de l'historique si `htmx.config.historyRestoreAsHxRequest` est désactivé.
HX-Target	L' id élément cible s'il existe
HX-Trigger-Name	L' name élément déclenché s'il existe
HX-Trigger	L' id élément déclenché s'il existe

En-têtes de réponse

htmx prend en charge certains en-têtes de réponse spécifiques à htmx :

- [HX-Location](#) - Permet d'effectuer une redirection côté client sans rechargement complet de la page
- [HX-Push-Url](#) - ajoute une nouvelle URL à l'historique
- [HX-Redirect](#) - peut être utilisé pour effectuer une redirection côté client vers un nouvel emplacement
- [HX-Refresh](#) Si cette option est définie sur « true », le client effectuera un rafraîchissement complet de la page.
- [HX-Replace-Url](#) - remplace l'URL actuelle dans la barre d'adresse
- [HX-Reswap](#) - vous permet de spécifier comment la réponse sera échangée. Voir [hx-swap](#) pour les valeurs possibles
- [HX-Retarget](#) - un sélecteur CSS qui met à jour la cible de la mise à jour de contenu vers un élément différent de la page
- [HX-Reselect](#) - Un sélecteur CSS qui vous permet de choisir quelle partie de la réponse doit être insérée. Remplace un sélecteur existant [hx-select](#) sur l'élément déclencheur.
- [HX-Trigger](#) - vous permet de déclencher des événements côté client
- [HX-Trigger-After-Settle](#) - vous permet de déclencher des événements côté

client après l'étape de règlement

- [HX-Trigger-After-Swap](#) - vous permet de déclencher des événements côté client après l'étape d'échange

Pour plus d'informations sur les [HX-Trigger](#) en-têtes, consultez [HX-Trigger la section En-têtes de réponse](#) .

L'envoi d'un formulaire via HTML présente l'avantage de s'affranchir du [schéma POST/Redirect/GET](#) . Après le traitement réussi d'une requête POST sur le serveur, il n'est plus nécessaire de renvoyer une [redirection HTTP 302](#). Le fragment HTML peut être renvoyé directement.

De plus, les en-têtes de réponse mentionnés ci-dessus ne sont pas transmis à htmx pour traitement avec les codes de redirection 3xx, tels que [HTTP 302 \(Redirection\)](#) . Le navigateur intercepte la redirection en interne et renvoie les en-têtes et la réponse de l'URL redirigée. Dans la mesure du possible, utilisez des codes de réponse alternatifs, comme 200, afin de permettre le retour de ces en-têtes.

Demande d'ordre des opérations

L'ordre des opérations dans une requête htmx est le suivant :

- L'élément est déclenché et lance une requête
 - Les valeurs sont collectées pour la requête
 - La `htmx-request` classe est appliquée aux éléments appropriés
 - La requête est ensuite émise de manière asynchrone via AJAX.
 - Lorsqu'une réponse est reçue, l'élément cible est marqué avec la `htmx-swapping` classe
 - Un délai d'échange optionnel est appliqué (voir l'attribut [hx-swap](#)).
 - L'échange de contenu est terminé.
 - la `htmx-swapping` classe est supprimée de la cible
 - La `htmx-added` classe est ajoutée à chaque nouveau contenu.
 - la `htmx-settling` classe est appliquée à la cible
 - Un délai de stabilisation est appliqué (par défaut : 20 ms).
 - Le DOM est réglé
 - la `htmx-settling` classe est supprimée de la cible
 - La `htmx-added` classe est supprimée de chaque nouveau contenu.

Vous pouvez utiliser les classes `htmx-swapping`and`` pour créer [des transitions CSS](#) entre les pages. `htmx-settling`

Validation

Htmx s'intègre à l' [API de validation HTML5](#) et n'envoie aucune requête de formulaire si un champ valide est invalide. Ceci est valable aussi bien pour les requêtes AJAX que pour les envois WebSocket.

Htmx déclenche des événements liés à la validation qui peuvent être utilisés pour intégrer une validation personnalisée et une gestion des erreurs :

- `htmx:validation:validate` - Appelée avant `checkValidity()` l'appel de la méthode d'un élément. Peut servir à ajouter une logique de validation personnalisée.
- `htmx:validation:failed` - appelé lorsque `checkValidity()` la fonction renvoie la valeur « faux », indiquant une entrée invalide
- `htmx:validation:halted` - appelée lorsqu'une requête n'est pas émise en raison d'erreurs de validation. Les erreurs spécifiques peuvent être trouvées dans l' `event.detail.errors` objet.

Par défaut, les éléments qui ne constituent pas un formulaire ne sont pas validés avant d'effectuer des requêtes, mais vous pouvez activer la validation en définissant l' `hx-validate` attribut sur « true ».

Lors de la soumission d'un formulaire via un navigateur classique, l'utilisateur est automatiquement averti des erreurs de validation et le focus est automatiquement placé sur le premier champ invalide. Pour des raisons de compatibilité avec les versions précédentes, htmx ne signale pas la validation par défaut ; il est donc recommandé d'activer cette option `htmx.config.reportValidityOfForms` pour `true` rétablir le comportement par défaut du navigateur.

Exemple de validation

Voici un exemple d'entrée qui utilise l' `hx-on` attribut pour intercepter l' `htmx:validation:validate` événement et exiger que l'entrée ait la valeur `foo` :

```
<form id="example-form" hx-post="/test">
  <input name="example"
    onkeyup="this.setCustomValidity('') // reset the validation on keyup"
    hx-on:htmx:validation:validate="if(this.value != 'foo') {
      this.setCustomValidity('Please enter the value foo') // set the va
      htmx.find('#example-form').reportValidity() // report the
    }">
</form>
```

Notez que toutes les validations côté client doivent être refaites côté serveur, car elles peuvent toujours être contournées.

Htmx vous permet d'utiliser **les transitions CSS** dans de nombreuses situations en utilisant uniquement HTML et CSS.

Extensions

Extensions principales

- [head-support](#) - prise en charge de la fusion des informations des balises head (styles, etc.) dans les requêtes htmx
- [htmx-1-compat](#) - restaure les paramètres et fonctionnalités par défaut de htmx 1
- [idiomorph](#) - prend en charge la morph stratégie d'échange utilisant idiomorph
- [préchargement](#) - permet de précharger du contenu pour de meilleures performances
- [response-targets](#) - vous permet de cibler des éléments en fonction des codes de réponse HTTP (par exemple 404)
- [sse](#) - prise en charge [des événements envoyés par le serveur](#)
- [ws](#) - prise en charge des [WebSockets](#)

Installation des extensions

```
<head>  
  <script src="https://cdn.jsdelivr.net/npm/hmx.org@2.0.10/dist/hmx.min.js" integrity="sha384-9wXUWYtR6EgDzFqLQvTlGfPmZCkxH7BdK1eS+VnN1y3o3333333333333333" crossorigin="anonymous"></script>  
  <script src="https://cdn.jsdelivr.net/npm/hmx-ext-response-targets@2.0.4" integrity="sha384-9wXUWYtR6EgDzFqLQvTlGfPmZCkxH7BdK1eS+VnN1y3o3333333333333333" crossorigin="anonymous"></script>  
</head>
```

```
<body hx-ext="extension-name">
  ...
```

Une version non minifiée est également disponible à l'adresse

<https://cdn.jsdelivr.net/npm/htmx-ext-extension-name/dist/extension-name.js> (remplacez `extension-name` par le nom de l'extension).

Bien que l'utilisation d'un CDN soit simple, il est déconseillé [de l'utiliser en production](#).

Une autre méthode simple pour installer les extensions htmx consiste à les copier directement dans votre projet. Téléchargez l'extension depuis

<https://cdn.jsdelivr.net/npm/htmx-ext-extension-name> [`extension-name` nom de l'extension] (par exemple : <https://cdn.jsdelivr.net/npm/htmx-ext-response-targets>). Ajoutez-la ensuite au répertoire approprié de votre projet et incluez-la aux endroits nécessaires à l'aide d'une `<script>` balise.

Pour les systèmes de construction de type npm, vous pouvez installer des extensions htmx via [npm](#) (remplacez `extension-name` par le nom de l'extension) :

```
npm install htmx-ext-extension-name
```

Après l'installation, vous devrez utiliser les outils appropriés pour l'intégration

`node_modules/htmx-ext-extension-name/dist/extension-name.js` (ou `.min.js`). Par exemple, vous pouvez intégrer l'extension avec le noyau htmx `node_modules/htmx.org/dist/htmx.js` et le code spécifique au projet.

Si vous utilisez un outil de regroupement pour gérer votre JavaScript (par exemple Webpack, Rollup) :

- Installez `htmx.org` -le `htmx-ext-extension-name` via npm (remplacez `extension-name` par le nom de l'extension).
- Importez les deux packages dans votre `index.js`

```
import `htmx.org`;
import `htmx-ext-extension-name`; // replace `extension-name` with the name of the ext
```

Remarque : [Idiomorph](#) ne suit pas la convention de nommage des extensions htmx.

Utilisez `idiomorph` plutôt que `htmx-ext-idiomorph`. Par exemple, ``htmx``

<https://cdn.jsdelivr.net/npm/idiomorph> ou `npm install idiomorph `htmx``.

Remarque : Les extensions communautaires hébergées en dehors de ce dépôt peuvent avoir des instructions d'installation différentes. Veuillez consulter le dépôt correspondant pour obtenir des instructions d'installation.

Activation des extensions

Pour activer une extension, ajoutez un `hx-ext="extension-name"` attribut à `<body>` un élément HTML (remplacez `extension-name` `<extension>` par le nom de l'extension). L'extension sera appliquée à tous les éléments enfants.

L'exemple suivant montre comment activer l'extension [response-targets](#), vous permettant de spécifier différents éléments cibles à échanger en fonction du code de réponse HTTP.

```
<body hx-ext="response-targets">
  ...
  <button hx-post="/register" hx-target="#response-div" hx-target-404="#not-found">
    Register!
  </button>
  <div id="response-div"></div>
  <div id="not-found"></div>
  ...
</body>
```

Création d'extensions

Si vous souhaitez ajouter votre propre extension à htmx, veuillez [consulter la documentation relative aux extensions](#).

Événements et journalisation

Htmx dispose d'un [mécanisme d'événements](#) étendu, qui sert également de système de journalisation.

Si vous souhaitez vous inscrire à un événement htmx donné, vous pouvez utiliser

```
document.body.addEventListener('htmx:load', function(evt) {
  myJavascriptLib.init(evt.detail.elt);
});
```

ou, si vous préférez, vous pouvez utiliser l'assistant HTML suivant :

```
htmx.on("htmx:load", function(evt) {
  myJavascriptLib.init(evt.detail.elt);
});
```

Cet `htmx:load` événement est déclenché chaque fois qu'un élément est chargé dans le DOM par htmx, et est en fait équivalent à l' `load` événement normal.

Voici quelques utilisations courantes des événements htmx :

Initialiser une bibliothèque tierce avec des événements

L'utilisation de cet `htmx:load` événement pour initialiser le contenu est si courante que htmx fournit une fonction d'assistance :

```
htmx.onLoad(function(target) {  
  myJavascriptLib.init(target);  
});
```

Cela fait la même chose que le premier exemple, mais c'est un peu plus propre.

Configurer une requête avec des événements

Vous pouvez gérer cet `htmx:configRequest` événement afin de modifier une requête AJAX avant son émission :

```
document.body.addEventListener('htmx:configRequest', function(evt) {  
  evt.detail.parameters['auth_token'] = getAuthToken(); // add a new parameter into  
  evt.detail.headers['Authentication-Token'] = getAuthToken(); // add a new header i  
});
```

Ici, nous ajoutons un paramètre et un en-tête à la requête avant son envoi.

Modification du comportement d'échange par les événements

Vous pouvez gérer cet `htmx:beforeSwap` événement afin de modifier le comportement d'échange de htmx :

```
document.body.addEventListener('htmx:beforeSwap', function(evt) {  
  if(evt.detail.xhr.status === 404){  
    // alert the user when a 404 occurs (maybe use a nicer mechanism than alert());  
    alert("Error: Could Not Find Resource");  
  } else if(evt.detail.xhr.status === 422){  
    // allow 422 responses to swap as we are using this as a signal that  
    // a form was submitted with bad data and want to rerender with the  
    // errors  
    //  
    // set isError to false to avoid error logging in console  
    evt.detail.shouldSwap = true;  
    evt.detail.isError = false;  
  } else if(evt.detail.xhr.status === 418){  
    // if the response code 418 (I'm a teapot) is returned, retarget the  
    // content of the response to the element with the id `teapot`  
    evt.detail.shouldSwap = true;  
    evt.detail.target = htmx.find("#teapot");  
  }  
});
```

Ici, nous gérons quelques [codes de réponse d'erreur de niveau 400](#) qui, normalement, n'entraîneraient pas d'échange en htmx.

Nommer les événements

Notez que tous les événements sont déclenchés avec deux noms différents.

- Étui Camel
- Kebab Case

Ainsi, par exemple, vous pouvez écouter `<br> htmx:afterSwap` ou `<c> htmx:after-swap`. Cela facilite l'interopérabilité avec d'autres bibliothèques. [Alpine.js](#), par exemple, exige la notation kebab-case.

Enregistrement

Si vous configurez un enregistreur de journalisation `htmx.logger`, chaque événement sera consigné. Cela peut s'avérer très utile pour le débogage :

```
htmx.logger = function(elt, event, data) {  
  if(console) {  
    console.log(event, elt, data);  
  }  
}
```

Débogage

La programmation déclarative et événementielle avec htmx (ou tout autre langage déclaratif) peut être une activité formidable et très productive, mais un inconvénient par rapport aux approches impératives est qu'elle peut être plus difficile à déboguer.

Par exemple, il peut être difficile de comprendre pourquoi quelque chose *ne se produit* pas si l'on ne connaît pas les astuces.

Voici les astuces :

Le premier outil de débogage que vous pouvez utiliser est la `htmx.logAll()` méthode `consign`. Celle-ci enregistrera chaque événement déclenché par htmx et vous permettra de voir exactement ce que fait la bibliothèque.

```
htmx.logAll();
```

Bien sûr, cela ne vous expliquera pas pourquoi le HTML *ne fonctionne* pas. Vous ignorez peut-être aussi *quels* événements un élément DOM déclenche. Pour résoudre ce problème, vous pouvez utiliser la `monitorEvents()` méthode disponible dans la console du navigateur :

```
monitorEvents(htmx.find("#theElement"));
```

Cela affichera `theElement` dans la console tous les événements qui se produisent sur l'élément ayant l'identifiant spécifié, et vous permettra de voir exactement ce qui se passe.

Notez que cela ne fonctionne *que* depuis la console ; vous ne pouvez pas l'intégrer dans une balise script sur votre page.

En dernier recours, vous pouvez tenter de déboguer `htmx.js` en chargeant la version non minifiée. Elle contient environ 2 500 lignes de JavaScript, ce qui représente une quantité de code tout à fait gérable. Il vous faudra probablement placer un point d'arrêt dans les méthodes `issueAjaxRequest()` et `handleAjaxResponse()` pour observer le comportement du programme.

Et n'hésitez pas à nous rejoindre sur [Discord](#) si vous avez besoin d'aide.

Création de démos

Parfois, pour illustrer un bug ou clarifier une utilisation, il est pratique de pouvoir utiliser un site de snippets JavaScript comme [jsfiddle](#) . Pour faciliter la création de démos, htmx héberge un site de scripts de démonstration qui installera :

- htmx
- hyperscript
- une bibliothèque de simulation de requêtes

Il vous suffit d'ajouter la balise script suivante à votre démo/fiddle/etc. :

```
<script src="https://demo.htmx.org"></script>
```

Cet outil vous permet d'ajouter des réponses simulées en insérant `template` des balises avec un `url` attribut indiquant l'URL. La réponse pour cette URL correspondra au contenu HTML interne du modèle, ce qui simplifie la création de réponses simulées. Vous pouvez ajouter un délai à la réponse grâce à un `delay` attribut, qui doit être un entier indiquant le nombre de millisecondes à attendre.

Vous pouvez intégrer des expressions simples dans le modèle en utilisant la `{{ }}` syntaxe suivante.

Notez que ceci ne doit être utilisé que pour des démonstrations et qu'il n'est en aucun cas garanti de fonctionner sur de longues périodes, car il récupérera toujours les dernières versions de htmx et hyperscript !

Exemple de démonstration

Voici un exemple du code en action :

```
<!-- Load demo environment -->
<script src="https://demo.htmx.org"></script>
```

```
<!-- post to /foo -->
<button hx-post="/foo" hx-target="#result">
  Count Up
</button>
<output id="result"></output>

<!-- respond to /foo with some dynamic content in a template tag -->
<script>
  globalInt = 0;
</script>
<template url="/foo" delay="500"> <!-- note the url and delay attributes -->
  ${globalInt++}
</template>
```

Scripting

Bien que htmx encourage une approche hypermédia pour le développement d'applications web, il offre de nombreuses options pour le script côté client. Le script est inclus dans la description RESTful de l'architecture web (voir : [Code à la demande](#)) . Dans la mesure du possible, nous recommandons une approche [hypermédia](#) pour le script dans votre application web.

- [Respectez HATEOAS](#)
- [Utilisez les événements pour assurer la communication entre les composants.](#)
- [Utilisez des îles pour isoler les composants non hypermédia du reste de votre application.](#)
- [Envisagez l'utilisation de scripts intégrés.](#)

Le principal point d'intégration entre htmx et les solutions de script réside dans les [événements](#) qu'htmx envoie et auxquels il peut répondre. Consultez l'exemple SortableJS dans la section [JavaScript tierce](#) pour un modèle pertinent d'intégration d'une bibliothèque JavaScript avec htmx via les événements.

Les solutions de script qui s'associent bien avec htmx incluent :

- [VanillaJS](#) – L'utilisation simple des fonctionnalités intégrées de JavaScript pour intercepter les événements émis par htmx est une solution très efficace pour le développement de scripts. Cette approche, extrêmement légère, est de plus en plus populaire.
- [AlpineJS](#) – Alpine.js offre un ensemble complet d'outils pour la création de scripts front-end sophistiqués, incluant la prise en charge de la programmation réactive, tout en restant extrêmement léger. Alpine encourage l'approche du « scripting en ligne » qui, selon nous, s'accorde parfaitement avec HTMLx.

- [jQuery](#) - Malgré son ancienneté et sa réputation dans certains milieux, jQuery s'associe très bien avec htmx, notamment dans les bases de code plus anciennes qui contiennent déjà beaucoup de jQuery.
- [Hyperscript](#) est un langage de script front-end expérimental créé par la même équipe que celle qui a développé htmx. Il est conçu pour s'intégrer parfaitement au HTML et permet à la fois de gérer et de créer des événements. Il s'associe très bien avec htmx.

Notre [livre](#) consacre un chapitre entier, intitulé « [Programmation côté client](#) », à la manière dont la programmation peut être intégrée à votre application basée sur HTMLx.

Les `hx-on*` attributs

HTML permet l'intégration de scripts en ligne via les [onevent propriétés](#), telles que `onClick` :

```
<button onclick="alert('You clicked me!')">
  Click Me!
</button>
```

Cette fonctionnalité permet de placer la logique de script au même endroit que les éléments HTML auxquels elle s'applique, assurant ainsi une bonne [localisation des comportements \(LoB\)](#). Malheureusement, HTML n'autorise `on*` les attributs que pour un nombre limité d'[événements DOM spécifiques](#) (par exemple, `on` onclick``) et ne propose pas de mécanisme général pour répondre à des événements arbitraires sur les éléments.

Pour pallier cette lacune, htmx propose `hx-on*` des attributs. Ces attributs permettent de réagir à tout événement tout en préservant le niveau de service des `on*` propriétés standard.

Si nous voulions répondre à l' `click` événement en utilisant un `hx-on` attribut, nous écririons ceci :

```
<button hx-on:click="alert('You clicked me!')">
  Click Me!
</button>
```

Donc, la chaîne de caractères `hx-on`, suivie de deux points (ou d'un tiret), puis du nom de l'événement.

Pour un `click` événement, nous recommandons bien sûr d'utiliser l' `onclick` attribut standard. Cependant, prenons l'exemple d'un bouton HTML qui souhaite ajouter un paramètre à une requête via l' `htmx:config-request` événement. Ceci serait impossible

avec une `on*` propriété standard, mais possible avec l' `hx-on:htmx:config-request` attribut :

```
<button hx-post="/example"
        hx-on:htmx:config-request="event.detail.parameters.example = 'Hello Scripting!'
        Post Me!
</button>
```

Ici, le `example` paramètre est ajouté à la `POST` requête avant son émission, avec la valeur « Hello Scripting ! ».

Un autre cas d'utilisation consiste à [réinitialiser les entrées utilisateur](#) lors de requêtes réussies à l'aide de l' `afterRequest` événement, évitant ainsi le recours à une opération de permutation hors bande.

Les `hx-on*` attributs constituent un mécanisme très simple pour l'exécution de scripts embarqués généralisés. Ils ne remplacent *pas* les solutions de script front-end plus complètes telles qu'AlpineJS ou Hyperscript. Ils peuvent toutefois enrichir une approche basée sur VanillaJS pour l'exécution de scripts dans votre application utilisant HTML5.

Notez que les attributs HTML ne sont *pas sensibles à la casse* . Cela signifie que, malheureusement, les événements qui dépendent de la casse (`camelCase`) ne peuvent pas être traités. Si vous devez gérer des événements en `camelCase`, nous vous recommandons d'utiliser une solution de script plus complète comme AlpineJS ou hyperscript. C'est précisément pour cette raison que htmx distribue tous ses événements en `camelCase` et en `kebab-case`.

JavaScript tiers

Htmx s'intègre assez bien avec les bibliothèques tierces. Si la bibliothèque déclenche des événements sur le DOM, vous pouvez utiliser ces événements pour déclencher des requêtes depuis htmx.

Un bon exemple en est la [démonstration SortableJS](#) :

```
<form class="sortable" hx-post="/items" hx-trigger="end">
  <div class="htmx-indicator">Updating...</div>
  <div><input type='hidden' name='item' value='1' />Item 1</div>
  <div><input type='hidden' name='item' value='2' />Item 2</div>
  <div><input type='hidden' name='item' value='2' />Item 3</div>
</form>
```

Avec Sortable, comme avec la plupart des bibliothèques JavaScript, il faut initialiser le contenu à un moment donné.

En jQuery, vous pourriez procéder comme suit :

```
$(document).ready(function() {
  var sortables = document.body.querySelectorAll(".sortable");
```

```

    for (var i = 0; i < sortables.length; i++) {
      var sortable = sortables[i];
      new Sortable(sortable, {
        animation: 150,
        ghostClass: 'blue-background-class'
      });
    }
  });
});

```

En HTML, vous utiliseriez plutôt la `htmx.onLoad` fonction et vous ne sélectionneriez que le contenu nouvellement chargé, plutôt que l'intégralité du document :

```

htmx.onLoad(function(content) {
  var sortables = content.querySelectorAll(".sortable");
  for (var i = 0; i < sortables.length; i++) {
    var sortable = sortables[i];
    new Sortable(sortable, {
      animation: 150,
      ghostClass: 'blue-background-class'
    });
  }
});

```

Cela permettra de garantir que, lors de l'ajout de nouveau contenu au DOM par htmx, les éléments triables soient correctement initialisés.

Si JavaScript ajoute au DOM du contenu comportant des attributs htmx, vous devez vous assurer que ce contenu est initialisé avec la `htmx.process()` fonction appropriée.

Par exemple, si vous récupériez des données et les insériez dans un élément div à l'aide de l' `fetch` API, et que ce code HTML contenait des attributs htmx, vous devriez ajouter un appel comme `htmx.process()` celui-ci :

```

let myDiv = document.getElementById('my-div')
fetch('http://example.com/movies.json')
  .then(response => response.text())
  .then(data => { myDiv.innerHTML = data; htmx.process(myDiv); } );

```

Certaines bibliothèques tierces créent du contenu à partir d'éléments de modèles HTML. Par exemple, Alpine JS utilise l' `x-if` attribut ``content`` sur les modèles pour ajouter du contenu de manière conditionnelle. Ces modèles ne font pas initialement partie du DOM et, s'ils contiennent des attributs ``htmx``, un appel à ``container`` sera nécessaire `htmx.process()` après leur chargement. L'exemple suivant utilise `$watch` la fonction d'Alpine pour détecter un changement de valeur qui déclencherait l'affichage de contenu conditionnel :

```

<div x-data="{show_new: false}"
  x-init="$watch('show_new', value => {
    if (show_new) {
      htmx.process(document.querySelector('#new_content'))
    }
  })"

```

```

    }
  })">
  <button @click = "show_new = !show_new">Toggle New Content</button>
  <template x-if="show_new">
    <div id="new_content">
      <a hx-get="/server/newstuff" href="#">New Clickable</a>
    </div>
  </template>
</div>

```

Composants Web

Veuillez consulter la page [Exemples de composants Web](#) pour des exemples d'intégration de HTML avec des composants Web.

Mise en cache

htmx fonctionne nativement avec les mécanismes [de mise en cache HTTP standard](#).

Si votre serveur ajoute l' [Last-Modified](#) en-tête de réponse HTTP à la réponse pour une URL donnée, le navigateur ajoutera automatiquement l' [If-Modified-Since](#) en-tête HTTP de requête aux requêtes suivantes adressées à cette même URL. Notez que si votre serveur peut afficher un contenu différent pour une même URL en fonction d'autres en-têtes, vous devez utiliser l' [Vary](#) en-tête HTTP de réponse. Par exemple, si votre serveur affiche le code HTML complet lorsque l' `HX-Request` en-tête est absent ou `false`, et un fragment de ce code HTML lorsque `HX-Request: true`, vous devez l'ajouter `Vary: HX-Request`. Cela permet au cache d'être indexé en fonction de l'URL de réponse et de l' `HX-Request` en-tête de requête, et non plus uniquement de l'URL de réponse. Désactivez toujours `htmx.config.historyRestoreAsHxRequest` l'option `--request-content` afin que les requêtes HTML complètes ne soient pas mises en cache avec des réponses partielles.

Si vous ne pouvez pas (ou ne souhaitez pas) utiliser l' `Vary` en-tête, vous pouvez également définir le paramètre de configuration `getCacheBusterParam` sur `true`. Si cette variable de configuration est définie, htmx inclura un paramètre de contournement du cache dans `GET` les requêtes qu'il effectue, ce qui empêchera les navigateurs de mettre en cache les réponses basées sur htmx et les réponses non basées sur htmx dans le même emplacement de cache.

Le format HTML fonctionne également [ETag](#) comme prévu. Notez que si votre serveur peut afficher un contenu différent pour une même URL (par exemple, selon la valeur de l' `HX-Request` en-tête), il doit générer une URL différente `ETag` pour chaque contenu.

Sécurité

Le format HTML vous permet de définir la logique directement dans votre DOM. Cela présente plusieurs avantages, le principal étant [la localité du comportement](#), qui rend votre système plus facile à comprendre et à maintenir.

Un problème lié à cette approche est toutefois la sécurité : puisque htmx augmente l'expressivité du HTML, si un utilisateur malveillant parvient à injecter du HTML dans votre application, il peut exploiter cette expressivité de htmx à des fins malveillantes.

Règle 1 : Échapper tout le contenu utilisateur

La première règle du développement web en HTML a toujours été : *ne faites pas confiance aux données saisies par l'utilisateur*. Vous devez échapper tout contenu tiers non fiable injecté sur votre site. Ceci afin de prévenir, entre autres, [les attaques XSS](#).

[Le site web de l'OWASP](#) propose une documentation exhaustive sur les attaques XSS et sur la manière de les prévenir, notamment une [fiche récapitulative sur la prévention des attaques Cross-Site Scripting](#).

La bonne nouvelle, c'est que ce sujet est très ancien et bien compris, et que la grande majorité des langages de modèles côté serveur prennent en charge [l'échappement automatique](#) du contenu pour éviter précisément ce genre de problème.

Cela dit, il arrive que certains choisissent d'injecter du HTML de manière plus risquée, souvent via un `raw()` mécanisme spécifique de leur langage de modèles. Cette pratique peut se justifier, mais si le contenu injecté provient d'un tiers, il *doit* être nettoyé, notamment en supprimant les attributs commençant par `<script>` `hx-` et `data-hx` `</script>`, ainsi que `<script>` les balises en ligne, etc.

Si vous injectez du code HTML brut et que vous gérez vous-même l'échappement des caractères, il est recommandé de *mettre sur liste blanche* les attributs et les balises autorisés, plutôt que de mettre sur liste noire ceux que vous interdisez.

Outils de sécurité htmx

Bien sûr, les bugs surviennent et les développeurs ne sont pas parfaits, il est donc judicieux d'adopter une approche multicouche de la sécurité de votre application web, et htmx fournit également des outils pour vous aider à la sécuriser.

Jetons-y un coup d'œil.

hx-disable

Le premier outil fourni par htmx pour renforcer la sécurité de votre application est l'[hx-disable](#) attribut `html_import`. Cet attribut empêche le traitement de tous les attributs htmx sur un élément donné, ainsi que sur tous les éléments qu'il contient. Par exemple, si vous incluiez du contenu HTML brut dans un modèle (ce qui est fortement déconseillé !), vous pourriez entourer ce contenu d'une balise `<div>` dotée `hx-disable` de cet attribut :

```
<div hx-disable>
  <%= raw(user_content) %>
</div>
```

Le composant HTML ne traitera aucun attribut ni fonctionnalité HTML présent dans ce contenu. Cet attribut ne peut être désactivé par l'ajout de contenu supplémentaire : si un `hx-disable` attribut est présent dans la hiérarchie parente d'un élément, il ne sera pas traité par HTML.

hx-history

Un autre aspect de sécurité à prendre en compte est le cache de l'historique HTML. Certaines pages peuvent contenir des données sensibles que vous ne souhaitez pas stocker dans le `localStorage` cache utilisateur. Vous pouvez exclure une page du cache de l'historique en incluant l' `hx-history` attribut `history_cache` n'importe où sur la page et en définissant sa valeur sur `false` .

Options de configuration

htmx propose également des options de configuration liées à la sécurité :

- `htmx.config.selfRequestsOnly` - Si cette option est activée `true` , seules les requêtes adressées au même domaine que le document actuel seront autorisées.
- `htmx.config.allowScriptTags` - htmx traitera `<script>` les balises trouvées dans le nouveau contenu qu'il charge. Si vous souhaitez désactiver ce comportement, vous pouvez définir cette variable de configuration sur `false`
- `htmx.config.historyCacheSize` - peut être configuré pour `0` éviter de stocker du code HTML dans le `localStorage` cache
- `htmx.config.allowEval` - peut être configuré pour `false` désactiver toutes les fonctionnalités htmx qui dépendent de `eval` :
 - filtres d'événements
 - `hx-on:` attributs
 - `hx-vals` avec le `js:` préfixe
 - `hx-headers` avec le `js:` préfixe

Notez que toutes les fonctionnalités supprimées par la désactivation `eval()` peuvent être réimplémentées à l'aide de votre propre code JavaScript personnalisé et du modèle d'événements htmx.

Événements

Si vous souhaitez autoriser les requêtes vers certains domaines autres que l'hôte actuel, sans pour autant les laisser totalement ouvertes, vous pouvez utiliser cet

`htmx:validateUrl` événement. Celui-ci contiendra l'URL de la requête `detail.url` et une `sameHost` propriété.

Vous pouvez examiner ces valeurs et, si la requête n'est pas valide, déclencher `preventDefault()` l'événement pour empêcher son émission.

```
document.body.addEventListener('htmx:validateUrl', function (evt) {
  // only allow requests to the current server as well as myserver.com
  if (!evt.detail.sameHost && evt.detail.url.hostname !== "myserver.com") {
    evt.preventDefault();
  }
});
```

Options CSP

Les navigateurs proposent également des outils pour renforcer la sécurité de votre application web. L'outil le plus puissant est une [politique de sécurité du contenu](#) (CSP). Grâce à une CSP, vous pouvez configurer le navigateur pour qu'il n'effectue pas de requêtes vers des hôtes autres que l'origine, qu'il n'interprète pas les balises de script intégrées, etc.

Voici un exemple de CSP dans une `meta` balise :

```
<meta http-equiv="Content-Security-Policy" content="default-src 'self';">
```

Cela indique au navigateur de n'autoriser que les connexions au domaine source d'origine. Cette configuration serait redondante `htmx.config.selfRequestsOnly`, mais une approche de sécurité multicouche est justifiée, voire idéale, pour la sécurité des applications.

Une discussion exhaustive des CSP dépasse le cadre de ce document, mais l' [article MDN](#) constitue un bon point de départ pour explorer ce sujet.

Prévention du CSRF

L'attribution et la vérification des jetons CSRF sont généralement du ressort du serveur, mais `htmx` il est possible de renvoyer automatiquement le jeton CSRF avec chaque requête grâce à l' `hx-headers` attribut ``CSRFToken``. Cet attribut doit être ajouté à l'élément ``<request>`` émettant la requête ou à l'un de ses éléments parents. Ainsi, les éléments ``<request>`` `html` et ``<request>`` constituent `body` des vecteurs globaux efficaces pour ajouter le jeton CSRF à l' HTTP en-tête de la requête, comme illustré ci-dessous.

Remarque : cette fonctionnalité `hx-boost` ne met pas à jour les balises `<img>` `<html>` ou `<script>`. Si vous l'utilisez avec une balise `<script>`, assurez-vous d'inclure le jeton CSRF dans un élément qui sera remplacé. De nombreux frameworks web

permettent d'insérer automatiquement le jeton CSRF comme champ caché dans les formulaires HTML. Cette pratique est recommandée. `<body> hx-boost`

```
<html lang="en" hx-headers='{ "X-CSRF-TOKEN": "CSRF_TOKEN_INSERTED_HERE" }'>
  :
</html>
```

```
<body hx-headers='{ "X-CSRF-TOKEN": "CSRF_TOKEN_INSERTED_HERE" }'>
  :
</body>
```

Les éléments ci-dessus sont généralement uniques dans un document HTML et devraient être faciles à localiser dans les modèles.

Configuration du HTML

Htmx propose des options de configuration accessibles par programmation ou de manière déclarative. Elles sont listées ci-dessous :

Variable de configuration	Info
<code>htmx.config.historyEnabled</code>	par défaut <code>true</code> , utile uniquement pour les tests
<code>htmx.config.historyCacheSize</code>	par défaut 10
<code>htmx.config.refreshOnHistoryMiss</code>	Par défaut <code>false</code> , si la valeur est définie sur <code>true</code> htmx, une actualisation complète de la page sera effectuée en cas d'absence d'historique, plutôt qu'une requête AJAX.
<code>htmx.config.defaultSwapStyle</code>	par défaut <code>innerHTML</code>
<code>htmx.config.defaultSwapDelay</code>	valeur par défaut 0
<code>htmx.config.defaultSettleDelay</code>	par défaut 20
<code>htmx.config.includeIndicatorStyles</code>	par défaut <code>true</code> (détermine si les styles d'indicateurs sont chargés)
<code>htmx.config.indicatorClass</code>	par défaut <code>htmx-indicator</code>
<code>htmx.config.requestClass</code>	par défaut <code>htmx-request</code>
<code>htmx.config.addedClass</code>	par défaut <code>htmx-added</code>
<code>htmx.config.settlingClass</code>	par défaut <code>htmx-settling</code>

Variable de configuration**Info**

<code>htmx.config.swappingClass</code>	par défaut <code>htmx-swapping</code>
<code>htmx.config.allowEval</code>	par défaut <code>true</code> , peut être utilisé pour désactiver l'utilisation de <code>eval</code> par htmx pour certaines fonctionnalités (par exemple, les filtres de déclenchement).
<code>htmx.config.allowScriptTags</code>	par défaut <code>true</code> , détermine si le HTML traitera les balises de script trouvées dans le nouveau contenu.
<code>htmx.config.inlineScriptNonce</code>	Par défaut <code>''</code> , aucun nonce ne sera ajouté aux scripts intégrés.
<code>htmx.config.attributesToSettle</code>	par défaut <code>["class", "style", "width", "height"]</code> , les attributs à régler pendant la phase de règlement
<code>htmx.config.inlineStyleNonce</code>	par défaut <code>''</code> , cela signifie qu'aucun nonce ne sera ajouté aux styles en ligne
<code>htmx.config.useTemplateFragments</code>	Par défaut <code>false</code> , les balises de modèle HTML sont utilisées pour analyser le contenu provenant du serveur (non compatible avec IE11 !).
<code>htmx.config.wsReconnectDelay</code>	par défaut <code>full-jitter</code>
<code>htmx.config.wsBinaryType</code>	par défaut <code>blob</code> , le type de données binaires reçues via la connexion WebSocket
<code>htmx.config.disableSelector</code>	Par défaut <code>[hx-disable], [data-hx-disable]</code> , htmx ne traitera pas les éléments possédant cet attribut ou un élément parent.
<code>htmx.config.withCredentials</code>	Par défaut <code>false</code> , autoriser les requêtes de contrôle d'accès intersites utilisant des informations d'identification telles que les cookies, les en-têtes d'autorisation ou les certificats clients TLS.

Variable de configuration**Info**

<code>htmx.config.timeout</code>	par défaut 0, le nombre de millisecondes que peut prendre une requête avant d'être automatiquement interrompue.
<code>htmx.config.scrollBehavior</code>	Par défaut, le comportement de défilement est « instantané » lors de l'utilisation du modificateur show <code>hx-swap</code> . Les valeurs autorisées sont : <code>instant</code> (le défilement doit être instantané en un seul saut), <code>smooth</code> (le défilement doit être animé de manière fluide) et <code>auto</code> (le comportement de défilement est déterminé par la valeur calculée de scroll-behavior).
<code>htmx.config.defaultFocusScroll</code>	Si l'élément sélectionné doit être affiché à l'écran, la valeur par défaut est <code>false</code> et peut être modifiée à l'aide du modificateur d'échange focus-scroll .
<code>htmx.config.getCacheBusterParam</code>	Par défaut, la valeur est <code>false</code> ; si elle est définie sur <code>true</code> , htmx ajoutera l'élément cible à la <code>GET</code> requête au format indiqué. <code>org.htmx.cache-buster=targetElementId</code>
<code>htmx.config.globalViewTransitions</code>	Si cette option est activée <code>true</code> , htmx utilisera l' API de transition de vue lors du remplacement du contenu.
<code>htmx.config.methodsThatUseUrlParams</code>	Par défaut <code>["get", "delete"]</code> , htmx formatera les requêtes utilisant ces méthodes en encodant leurs paramètres dans l'URL, et non dans le corps de la requête.
<code>htmx.config.selfRequestsOnly</code>	par défaut <code>true</code> , indique si les requêtes AJAX doivent uniquement être autorisées vers le même domaine que le document actuel.

Variable de configuration**Info**`htmx.config.ignoreTitle`

Par défaut, la valeur est `false`, si elle est définie sur `true` htmx, le titre du document ne sera pas mis à jour lorsqu'une `title` balise est trouvée dans un nouveau contenu.

`htmx.config.disableInheritance`

désactive l'héritage des attributs dans htmx, qui peut ensuite être remplacé par l' `hx-inherit` attribut

`htmx.config.scrollToViewOnBoost`

Par défaut, la valeur est `true`, que la cible d'un élément mis en avant soit ou non affichée à l'écran. Si l'attribut `hx-target` est omis pour un élément mis en avant, la cible est par défaut `body`, ce qui entraîne le défilement de la page vers le haut.

`htmx.config.triggerSpecsCache`

Par défaut `null`, le cache stocke les spécifications de déclenchement évaluées, ce qui améliore les performances d'analyse au prix d'une consommation de mémoire accrue. Vous pouvez définir un objet simple pour utiliser un cache qui ne s'efface jamais, ou implémenter votre propre système à l'aide d'un [objet proxy](#).

`htmx.config.responseHandling`

Le comportement par défaut [de gestion des réponses](#) pour les codes d'état de réponse peut être configuré ici pour inverser ou générer une erreur.

`htmx.config.allowNestedOobSwaps`

Par défaut `true`, cette valeur détermine s'il faut traiter les échanges OOB sur les éléments imbriqués dans l'élément de réponse principal. Voir [Échanges OOB imbriqués](#).

`htmx.config.historyRestoreAsHxRequest`

Par défaut `true`, cette option permet de traiter les requêtes de rechargement complet de page manquantes dans le

Variable de configuration

Info

cache d'historique comme des requêtes « HX-Request » en renvoyant cet en-tête de réponse. Cette option doit toujours être désactivée lorsque l'en-tête HX-Request est utilisé pour renvoyer éventuellement des réponses partielles.

Vous pouvez les définir directement en JavaScript, ou vous pouvez utiliser une meta balise :

```
<meta name="htmx-config" content='{ "defaultSwapStyle": "outerHTML" }'>
```

Conclusion

Et voilà !

Amusez-vous avec HTML ! Vous pouvez faire [beaucoup de choses](#) sans écrire des tonnes de code !

haïku

*Fatigue JavaScript :
envie d'un hypertexte
déjà en main*

[documents](#)
[référence](#)
[exemples](#)
[parler](#)
[essais](#)
[@htmx_org](#)

